

КУРСОВАЯ РАБОТА

**Создание мобильного приложения, реализующего планнер с
интеграцией учебного расписания
по дисциплине «Конструирование программного обеспечения»**

Выполнили
студенты гр. 5130904/30107

Л. А. Марков
П. О. Калашникова
А. Д. Абросимов
К. А. Байкин

Руководитель

В. А. Юркин

Оглавление

Введение.....	3
Основная часть	4
Этап №0. Подготовительный	4
Этап №1. Определение проблемы	5
Этап №2. Выработка требований.....	6
Этап №3. Разработка архитектуры и детальное планирование	7
Этап №4. Кодирование и отладка	18
Этап №5. Unit & интеграционное тестирование	19
Этап №6. Нагрузочное тестирование	20
Заключение	21
Список использованной литературы.....	22

Введение

Создание программного продукта для обширных масс может представляться довольно простым: написал код, исправил все баги и выпустил продукт в люди. Однако знающий человек, работавший в коллективах более 5 человек, знает, что одним из самых важных, а может и самым важным этапом в процессе создания продукта является именно этап конструирования.

Основная задача этой курсовой работы — пройти большинство классических стадий создания ПО, а уже во вторую очередь — создать приложение планнер.

Основная часть

Этап №0. Подготовительный

Прежде всего необходимо определить **технологический стек** проекта. Так как мы пишем мобильное приложение и под Android и под ios, а так же хотим использовать внешнюю базу данных, то технологический стек вырисовывается сам собой:

- ⑩ **IDE – Android Studio**
- ⑩ *Язык программирования (приложение) — Kotlin*
- ⑩ *Язык программирования (серверная часть) — Python*
- ⑩ *Framework для обрисовки пользовательского интерфейса — Jetpack Compose*
- ⑩ *Архитектура — Kotlin Multiplatform* (так как мы хотим мульти-девайсное приложение, а не только Android, который в ванильном Jetpack Compose)
- ⑩ *СУБД — PostgreSQL*
- ⑩ *Система контроля версий — GitHub*
- ⑩ *Внешние API – API расписания СПбПУ и Google calendar API*

Этап №1. Определение проблемы

На рынке приложений-планеров не существует продукта, реализующего удобную синхронизацию между учебным расписанием и повседневными задачами, соединяющего всю информацию об учебных и внеучебных активностях вуза, с возможностью установки на определенное время PUSH-уведомлений.

Этап №2. Выработка требований

Выработаем **требования** к продукту от лица пользователя:

- ⑩ Как студент Политеха, я хочу иметь возможность синхронизации учебного расписания с повседневными задачами, чтобы планировать весь свой день в одном приложении.
- ⑩ Как студент Политеха, я хочу всегда быть в курсе проходящих в стенах моего вуза событий, чтобы активно участвовать в жизни института.

Этап №3. Разработка архитектуры и детальное планирование

Определим характер нагрузки на сервис:

1. Соотношение R/W нагрузки:

Нагрузка не будет равномерно распределена в течение суток. Явные пики будут возникать:

- Утром (перед началом пар)
- В начале недели (понедельник)

Операция при запуске приложения:

- Чтение (R): Сервис должен прочитать из базы данных номер академической группы.
- Запись (W): Сервис делает запрос к внешнему API, получает расписание для этой группы и записывает его в базе данных.
- Чтение (R): Приложение читает свежее расписание из БД и показывает пользователю.

Вывод: На один запуск приложения приходится как минимум 2 операции чтения и 1 операция записи — 2:1 в пользу чтения.

2. Объемы трафика:

Пиковая нагрузка — 10 000 пользователей в сутки.

Допустим, 70% всех запусков (7000) происходит за 2 пиковых часа (7200 секунд). Тогда пиковый RPS = $7000 / 7200 \sim 1$ RPS. Это очень низкий показатель.

В реальности пик может быть короче. Допустим, 1000 пользователей запустят приложение за 5 минут (300 секунд) перед первой парой. Тогда пиковый RPS = $1000 / 300 \sim 3.3$ RPS.

Расчет объема данных:

Основной объем — это отдача расписания (~ 20 КБ на запрос). Пиковый исходящий трафик: $3.3 \text{ RPS} * 20 \text{ КБ} \sim 66 \text{ КБ/с} \sim 0.5 \text{ Мбит/с}$. Это ничтожно мало.

Вывод: Сетевой трафик для такого сервиса минимален и не является узким местом.

3. Объемы дисковой системы

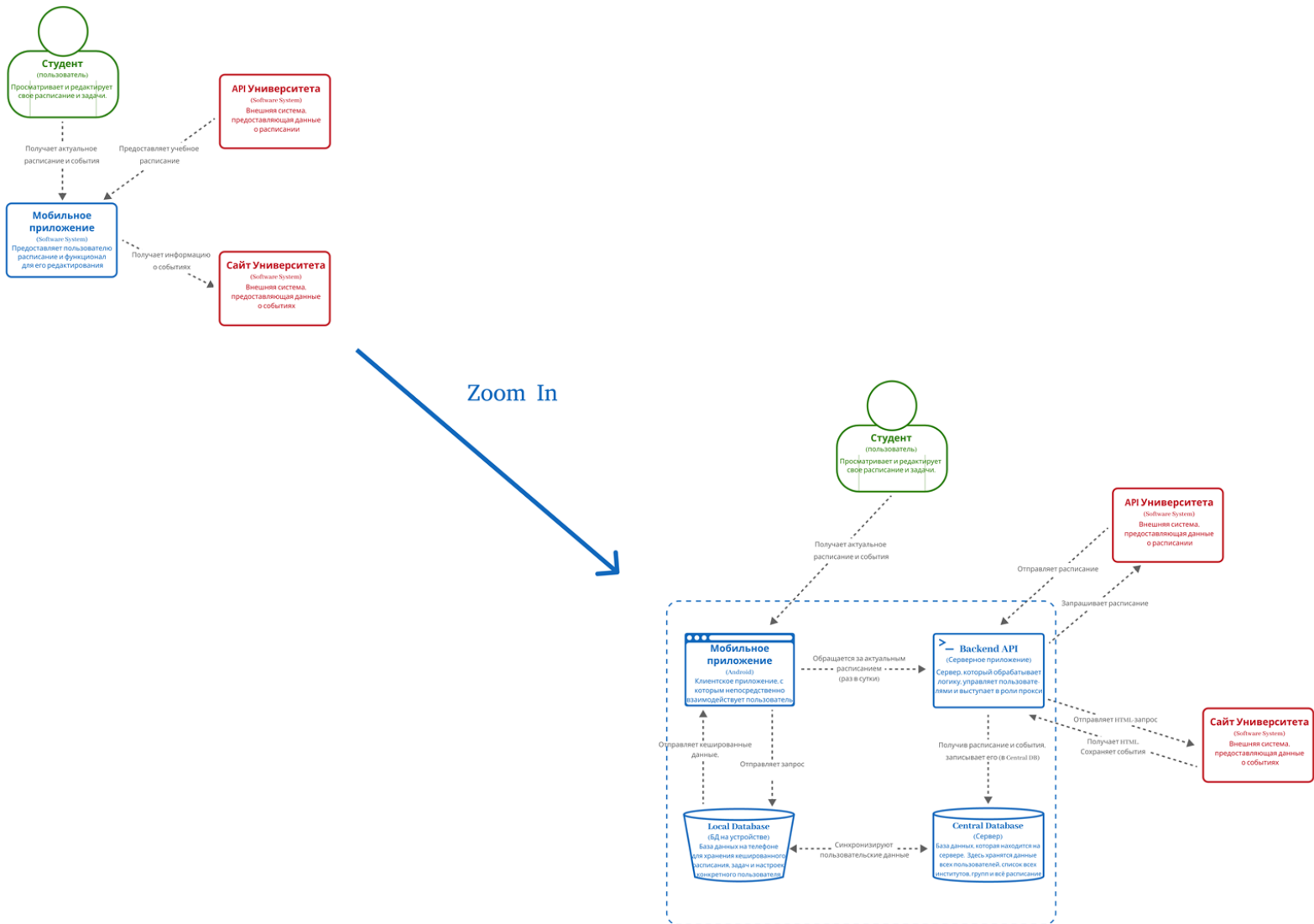
Основную часть дисковой системы будут занимать пары всех групп.

В среднем у каждой группы ~ 4 пары в день (берем на вырост). Количество учебных дней - 6, значит пар в неделю будет $6 * 4 = 24$ штуки.

В год ~ 45 учебных недель, значит $24 * 45 = 1080$ пар в год для одной группы. Количество институтов в Политехе - 16, групп в институте ~ 100 , значит всего групп будет $16 * 100 = 1600$ групп, тогда пар для всего Политеха $1080 * 1600 = 1\,728\,000$

Запись одной пары обойдется ~ в 200 байт, значит всего для хранения всех пар Политеха понадобится $1\,728\,000 * 200 = 0.3\text{ ГБ}$

Диаграммы архитектуры нашего приложения из подхода c4 model:



Определим основные **контракты API**:

1. Получение списка институтов

HTTP Method: GET

URL: <https://ruz.spbstu.ru/api/v1/ruz/faculties>

Path Params: -

Success response: 200

Response format: JSON

Response example:

```
{
  "faculties": [
    {
      "id": 121,
      "name": "Институт физической культуры, спорта и туризма",
      "abbr": "ИФКСиТ"
    },
    {
      "id": 123,
      "name": "Институт электроники и телекоммуникаций",
      "abbr": "ИЭиТ"
    }, {}
  ], {}
  ...
]
```

2. Получение списка групп института

HTTP Method: GET

URL: https://ruz.spbstu.ru/api/v1/ruz/faculties/{faculty_id}/groups

Path Params: {faculty_id} - id института

Success response: 200

Response format: JSON

Response example:

```
{
  "groups": [
    {
      "id": 42817,
      "name": "5130203/20102",
      "level": 4,
      "type": "common",
      "kind": 0,
      "spec": "",
      "year": 2025
    },
    {
      "id": 42789,
```

```

"name": "5130902/30202",
"level": 3,
"type": "common",
"kind": 0,
"spec": "",
"year": 2025
},
...
]
}

```

3. Поиск группы по номеру

Назначение: Достать id группы

HTTP Method: GET

URL: <https://ruz.spbstu.ru/api/v1/ruz/search/groups>

Query Params: q - поисковая строка (например: ?q=5130904/30107)

Success response: 200

Response format: JSON

Response example:

```

{
  "groups": [
    {
      "id": 42799,
      "name": "5130904/30107",
      "level": 3,
      "type": "common",
      "kind": 0,
      "spec": "",
      "year": 2025,
      "faculty": {
        "id": 125,
        "name": "Институт компьютерных наук и кибербезопасности",
        "abbr": "ИКНК"
      }
    }
  ]
}

```

4. Получение расписания на неделю

HTTP Method: GET

URL: https://ruz.spbstu.ru/api/v1/ruz/scheduler/{group_id}

Path Params: {group_id} - id группы

Query Params: date - опционально (в формате YYYY-MM-DD). Определяет неделю, в которую входит указанная дата и в качестве ответа отправляет расписание на эту неделю. Если не указана, возвращает расписание на текущую

неделю.

Success response: 200

Response format: JSON

Response example:

```
{
  "week": {
    "date_start": "2025.09.15",
    "date_end": "2025.09.21",
    "is_odd": true
  },
  "days": [
    {
      "weekday": 1,
      "date": "2025-09-15",
      "lessons": [
        {
          "subject": "Архитектура программных систем",
          "subject_short": "Архитектура программных систем",
          "type": 0,
          "additional_info": "",
          "time_start": "10:00",
          "time_end": "11:40",
          "parity": 0,
          "typeObj": {
            "id": 27,
            "name": "Практика",
            "abbr": "Пр"
          },
          "groups": [
            **Перечисление групп в формате п. 3**
          ],
          "teachers": [
            {
              "id": 22154,
              "oid": 34639,
              "full_name": "Гончаров Александр Викторович",
              "first_name": "Гончаров",
              "middle_name": "Александр",
              "last_name": "Викторович",
              "grade": "",
              "chair": "51/03 Высшая школа программной инженерии"
            }
          ],
          "auditoriums": [
            {
```

```

    "id": 894,
    "name": "104",
    "building": {
      "id": 18,
      "name": "3-й учебный корпус",
      "abbr": "3 к.",
      "address": ""
    }
  },
  "webinar_url": "",
  "lms_url": "https://dl.spbstu.ru/course/view.php?id=7151"
},
{ }, { } **Остальные занятия в этот день**
]
},
{ }, { } **Остальные дни в этой неделе**
]
}

```

Ожидаемые нефункциональные требования на время отклика:

Требования ко времени загрузки экрана UI:

- Старт приложения с нуля < 1.5 секунд
- Старт приложения из фона < 0.5 секунд

Требования к отзывчивости UI:

- Все анимации и переходы между экранами должны быть плавными и стабильными, обеспечивая ****60 FPS**** на среднем по мощности устройстве. На устройствах низкого ценового сегмента допустимо кратковременное падение до 30 FPS в моменты пиковой нагрузки
- Реакция на любое действие пользователя не должна превышать 100 мс

Требования к запросам API:

- Таймаут соединения - 3 секунды
- Таймаут чтения - 5 секунд
- Запросы на получение расписания обещают быть обработанными не более чем за 800 мс
- Запросы на получение массивных списков - не более чем за 1200 мс

Требования к работе с БД:

- Время доступа к локальной БД не должно занимать больше 50 мс и производиться асинхронно, чтобы не замедлять работу UI потока.

Данные требования на деле могут варьироваться в зависимости от устройства

пользователя, доступности интернет соединения, нагрузки на сервера <https://ruz.spbstu.ru> и не только.

Определим **схему внешней базы данных**:

Таблица ИНСТИТУТЫ с полями:

- ⑩ id (primary key)
- ⑩ name
- ⑩ abbr

Таблица ГРУППЫ с полями:

- ⑩ id (primary key)
- ⑩ faculty_id (связь МНОГО-ОДИН из таблицы ИНСТИТУТЫ)
- ⑩ name

Таблица РАСПИСАНИЕ с полями:

- ⑩ id (primary key - просто уникальный идентификатор занятия)
- ⑩ group_id (связь МНОГО-ОДИН из таблицы ГРУППЫ)
- ⑩ date
- ⑩ weekday (день недели от 1-пн до 6-суб)
- ⑩ subject
- ⑩ type (практика/лекция/лабораторная работа)
- ⑩ start_time
- ⑩ end_time
- ⑩ teacher
- ⑩ audithory
- ⑩ place (корпус)

Таблица ИВЕНТЫ с полями:

- ⑩ id (primary key)
- ⑩ title
- ⑩ description
- ⑩ date
- ⑩ start_time
- ⑩ end_time
- ⑩ location
- ⑩ creator
- ⑩ calendar_name

Определим **схему локальной базы данных**:

Таблица ИНСТИТУТЫ:

- ⑩ id (PRIMARY KEY)
- ⑩ name
- ⑩ abbr

Таблица ГРУППЫ:

- ⑩ id (PRIMARY KEY)
- ⑩ faculty_id
- ⑩ name

Таблица ИБЕНТЫ:

- ⑩ id (PRIMARY KEY)
- ⑩ title
- ⑩ description
- ⑩ date
- ⑩ start_time
- ⑩ end_time
- ⑩ location
- ⑩ creator
- ⑩ calendar_name
- ⑩ is_tracked
- ⑩ is_done

Таблица РАСПИСАНИЕ (но только группы пользователя):

- ⑩ id (PRIMARY KEY)
- ⑩ group_id
- ⑩ date
- ⑩ weekday
- ⑩ subject
- ⑩ type
- ⑩ start_time
- ⑩ end_time
- ⑩ teacher
- ⑩ audithory
- ⑩ place
- ⑩ is_done

Таблица ЗАМЕТКИ (только пользователя):

- ⑩ id (PRIMARY KEY)
- ⑩ date
- ⑩ place
- ⑩ header
- ⑩ note
- ⑩ is_notifications_enabled

- ⑩ start_time
- ⑩ end_time
- ⑩ is_done

Таблица СТИКЕРЫ:

- ⑩ id (PRIMARY KEY)
- ⑩ header
- ⑩ note

Таблица НАСТРОЙКИ:

- ⑩ key (PRIMARY KEY)
- ⑩ value

Она **выдержит нефункциональные требования**, так как вся база данных будет занимать не более 500 МБ, следовательно обращения к ней не будут долгими и колоритными.

Требования к загрузке UI будут удовлетворены технологическим стеком, а именно jetpack Compose и его хорошо-проработанный Composer. Требования к отзывчивости UI будут выполнены, используя compose-функции LazyRow и LazyColumn. Требования к работе с данными будут выдержаны через партиционирование, оптимизацию запросов, а так же в виду малого объема данных для хранения (500 МБ).

Схема масштабирования сервиса при росте нагрузки в 10 раз

Что изменится при росте:

При росте $\times 10$ количество запросов будет до 30-35 в секунду.

Объем базы: вместо ~500 МБ, станет ~5 ГБ.

Трафик: ~5-6 Мб/с (для сравнения - обычный домашний интернет легко держит больше 50).

Вывод: нагрузка все еще небольшая. Главное внимание к базе данных и обращениям к внешнему API расписаний.

Архитектура при росте Сервер приложения:

Сделать несколько копий (например 3-4) и поставить перед ними балансировщик, который будет распределять запросы между копиями.

Если нагрузка вырастает - автоматически поднимать больше копий. Если нагрузка падает - выключать лишние.

Базы данных Большая часть запросов - это именно чтение расписаний.

Поэтому необходимо:

- ⑩ Оставить одну базу данных для записей
- ⑩ Добавить реплики для чтения (1-2 штуки). Запросы на получение расписаний отправлять в реплики.
- ⑩ Использовать пул соединений, чтобы база данных не "захлебывалась" от множества коротких подключений.
- ⑩ Добавить кэширование
- ⑩ Хранить локально последние полученные расписания. При повторном запросе - брать от туда данные.

Взаимодействие с API:

- ⑩ Делать запросы заранее ночью или за несколько часов до пар, чтобы данные уже были в базе.
- ⑩ Если API недоступно, то брать расписание из кэша и показывать пользователю последнюю версию.
- ⑩ Как система будет работать при нагрузке

Пользователь открывает приложение.

- ⑩ Сервер проверяет кэш:
 - если данные есть - мгновенный ответ;
 - если нет - берёт из базы;
- ⑩ Новые данные в фоне подгружаются и обновляют кэш и базу данных.

Итог

Сервер: с 2 до 4 экземпляров, с возможностью автоматического увеличения при росте нагрузки.

База данных: добавить несколько реплик для чтения

Кэш: хранить расписания в памяти

Внешний API: данные подгружать заранее и кэшировать, чтобы не зависеть от задержек или ошибок

Этап №4. Кодирование и отладка

Для равномерного ведения работы, была введена система «тикетов» - была создана таблица, в которую в основном раз в неделю записывались *задания*, которые нужно сделать, любой участник команды мог взять любое задание и выполнить его. Если задание взято, то мягкий дедлайн для него — *неделя*. Так же в эту таблицу забивались данные о времени взятия и о самом человеке, который взял себе тикет.

Всю таблицу можно посмотреть по ссылке:

https://docs.google.com/spreadsheets/d/1luKtJpCuSXXXBYEjt_J1BARWbvWvGZ8ZIRS9bv_-1dsg/edit?gid=0#gid=0

Так же еженедельно без исключений во вторник в 20:00 проводилось общее собрание, на котором мы обсуждали проделанную за неделю работу, оперативно решали встающие перед нами вопросы и определяли тикеты на следующую неделю.

P.S.: *тикеты появлялись не только после собраний, но и в процессе разработки: как только кто-то увидел обширную, еще не сделанную работу, то сразу же оформлял ее в виде тикета, с подробным описанием проблемы, и тем, что необходимо сделать.*

Этап №5. Unit & интеграционное тестирование

Необходимо было покрыть весь код тестами. Из-за специфики нашего приложения, Unit и интеграционное тестирования объединены в один блок, так как покрытие рассчитывается суммарно, и нельзя обеспечить 90+% покрытия только Unit тестами без затрагивания интеграционных, и наоборот только интеграционными, без Unit тестов.

На этом этапе возникла довольно неприятная сложность в виде выбора библиотеки для тестирования. Дело в том, что обычные, привычные нам *junit-тесты* не поддерживаются в КМР (Kotlin Multiplatform) проектах, в связи со свежестью архитектуры КМР-подхода. Поэтому была использована специальная библиотека для *kotlin тестов*, от создателей самого языка kotlin. Это что касается обычного kotlin-кода.

Для тестов локальной БД необходимо тестировать специальные *suspend-функции*, которые реализованы через механизм «*корутин*» (*coroutines*) — сопрограммы или блоки кода, которые работают асинхронно и занимают поток только тогда, когда там действительно нужно что-то вычислять, в остальных же случаях, они попросту не занимают время потоков. Корутины нашли отличное применение в реализации запросов к БД. Их мы тестировали через специальную библиотеку *kotlinx-coroutines-test*.

Для интеграционных тестов, которые из-за нашей специфики объединены с UI тестированием, использовалась библиотека *compose-ui-test*. Здесь мы тестировали пользовательские *линейки событий* — «открыть главный экран, нажать на кнопку добавления заметки, выбрать время, написать заголовок, нажать сохранить». В процессе всех этих действий должна соответствующим образом меняться картинка на экране и после завершения линейки действий, должны измениться данные в локальной БД. Все это мы и тестировали.

По результатам тестов, мы добились покрытия кода в 88,5%, что является отличным результатом.

Этап №6. Нагрузочное тестирование

Осталось проверить всю систему под нагрузкой. Для этого будем использовать фреймворк “Locust”, который имитирует тысячи одновременных пользователей. Тест проводился для максимального количества в 1000 пользователей, когда сначала 0 пользователей и далее на каждой секунде появляется 50 пользователей. Тест показал количество RPS = 482, что является отличным показателем. Среднее время ожидания ответа сервера составило 62.55ms, что тоже говорит о том, что система стабильна.

Заключение

Было разработано приложение-планнер с интеграцией учебного расписания. Мы прошли классические этапы разработки ПО. Сейчас приложение отлично функционирует и полностью поддерживается на ОС Android. Так же возможно использование на IOS, но пока в режиме «demo».

Наша команда планирует и дальше продолжать работу над приложением, как минимум, потому что 3ое из 4х участников команды — владельцы гаджетов с ОС IOS. Мы не планируем распространение этого приложения в массы, а лишь хотим самим им пользоваться, что и делает его таким душевным и user-friendly-направленным.

Список использованной литературы

1. Макконнелл С. Совершенный код: мастер-класс / Пер. с англ. – М. : Русская редакция, 2017. – 896 с.
2. Android Developers: Compose Documentation // [Официальный сайт] / Google. – 2025. – URL: <https://developer.android.com/develop/ui/compose/documentation>. – (дата обращения: 20.09.2025 - 26.11.2025).
3. Android Developers: Compose UI Test Package Summary // [Официальный сайт] / Google. – 2025. – URL: <https://developer.android.com/reference/kotlin/androidx/compose/ui/test/package-summary>. – (дата обращения: 11.11.2025).